

25 Database Testing Q&A

Doc By Aston Cook

This guide covers core concepts, queries, and best practices for testing databases in QA projects.

1. What is database testing?

Answer: Database testing validates data integrity, accuracy, and consistency. It ensures backend data operations (CRUD, triggers, stored procedures, constraints) work as expected with frontend and API layers.

2. Why is database testing important for QA?

Answer: Applications rely on accurate data. Bugs in queries, migrations, or transactions can cause data loss, corruption, or incorrect outputs that may not show at the UI level.

3. What are CRUD operations in database testing?

Answer: CRUD stands for Create, Read, Update, Delete. QA engineers validate these core operations through direct SQL queries and application workflows.

4. How do you validate data integrity?

Answer: By checking primary keys, foreign keys, unique constraints, and ensuring relationships between tables are not broken. Example: no orphan orders without customers.

5. How do you test stored procedures?

Answer: Execute stored procedures with valid and invalid inputs, verify output correctness, check business logic, and test performance with large datasets.

6. How do you test triggers?

Answer: Insert or update records that should fire triggers. Validate that side effects occur correctly (e.g., audit logs are created, dependent records updated).

7. How do you validate transactions?

Answer: Test commit and rollback behavior. Example: simulate a payment failure and verify all related updates roll back without partial changes.

8. How do you test database migrations?

Answer: Run migrations in staging with real data. Verify schema updates, ensure backward compatibility, and confirm no data loss.

9. How do you test indexing and performance?

Answer: Use **EXPLAIN** or **EXPLAIN ANALYZE** to review query plans. Validate queries use indexes, measure response times, and check for full table scans.

10. What is normalization and why test it?

Answer: Normalization organizes data to reduce redundancy and improve integrity. QA ensures the schema follows normalization rules while balancing performance needs.

11. How do you test denormalized schemas?

Answer: Validate redundant data fields are consistent across tables. Test updates in one table propagate to dependent fields.

12. How do you validate referential integrity?

Answer: Run LEFT JOIN queries to detect orphan records. Example: orders without a valid customer ID.

13. How do you test data constraints?

Answer: Insert invalid values (e.g., null in a NOT NULL field, duplicate in a unique field) and confirm the database rejects them.

14. How do you test database security?

Answer: Validate role-based permissions, ensure least-privilege access, test injection prevention, and verify sensitive data is encrypted at rest and in transit.

15. How do you test backup and recovery?

Answer: Trigger backups, simulate failures, and attempt restores. Verify data completeness, consistency, and minimal downtime.

16. How do you test data consistency across environments?

Answer: Compare row counts, schema, and checksums between staging, QA, and production. Automate consistency checks during deployments.

17. How do you test ETL processes?

Answer: Validate source-to-target mappings, check row counts before and after, confirm transformations, and test edge cases like nulls or malformed data.

18. How do you test database partitioning?

Answer: Verify queries return correct results from partitioned tables. Test performance improvements and confirm data is routed to correct partitions.

19. How do you validate database auditing?

Answer: Perform operations that should create audit logs. Verify records include correct user IDs, timestamps, and actions.

20. How do you test for SQL injection vulnerabilities?

Answer: Input malicious payloads (' OR 1=1 --) and confirm the database is protected by parameterized queries and input validation.

21. How do you test database replication?

Answer: Insert data into the primary DB and confirm replication to replicas. Test failover scenarios and measure replication lag.

22. How do you test database sharding?

Answer: Validate data distribution across shards. Check queries route correctly and aggregations handle multi-shard data.

23. How do you validate data migrations between systems?

Answer: Perform record counts, spot checks, and reconciliation queries. Verify data mappings and transformations follow business rules.

24. How do you test database caching layers?

Answer: Confirm cache hits vs misses, validate updates invalidate cache, and test fallback when cache is unavailable.

25. What are best practices for database testing?

Answer:

- Automate common queries for faster checks.
- Use realistic test data, anonymized if needed.
- Monitor query performance and indexes.
- Test both schema and business logic.
- Validate across APIs, UI, and DB directly.